

Evolutionary Hyper Parameter Optimization to Find Lightweight CNN Models for Autonomous Steering

Devson Butani, Ryan Kaddis, CJ Chung

College of Arts & Sciences, Lawrence Technological University



ABSTRACT

This research investigates the optimization of Convolutional Neural Networks (CNNs) for self-driving car applications using the (n+m) Evolutionary Strategy (ES) with the 1/5th success rule. The objective is to develop a lightweight CNN model (small enough to run on cars) capable of real-time steering angle prediction, mimicking human driving behavior on a predefined path. The ES algorithm automates hyper parameter tuning, dynamically adjusting parameters such as learning rate and filter sizes. By employing the 1/5th success rule, mutation step sizes are adaptively scaled to balance exploration of the search space with the refinement of optimal solutions.

Data collection encompasses diverse driving scenarios recorded via the LTU ACTor autonomous driving platform. These include variations in path direction and driving style, enhancing model generalizability across environmental conditions. The dataset consists of timestamped images labeled with steering angles and pre-processed to focus on relevant visual information, such as the driving path.

Initial experiments involve training a baseline CNN model, which is then refined using ES to significantly reduce model size while maintaining competitive predictive accuracy. The results highlight the viability of lightweight CNN architectures for real-time autonomous systems, striking a balance between computational efficiency and performance. This study not only advances LTU's research initiatives but also lays the groundwork for deploying cost-effective, scalable solutions in self-driving technology.

INTRODUCTION

Autonomous driving systems require robust models capable of making real-time decisions under varying conditions. A critical component of these systems is the prediction of steering angles based on camera input, which involves processing visual data effectively and efficiently [3] on a compute limited on-board processor. This research focuses on optimizing CNNs leveraging evolutionary strategies (ES) to automate the search for the best-performing and best size-reduced model configurations. ES enables optimizing with high-dimensional search spaces (lots of unknown parameters) without requiring predefined search spaces.

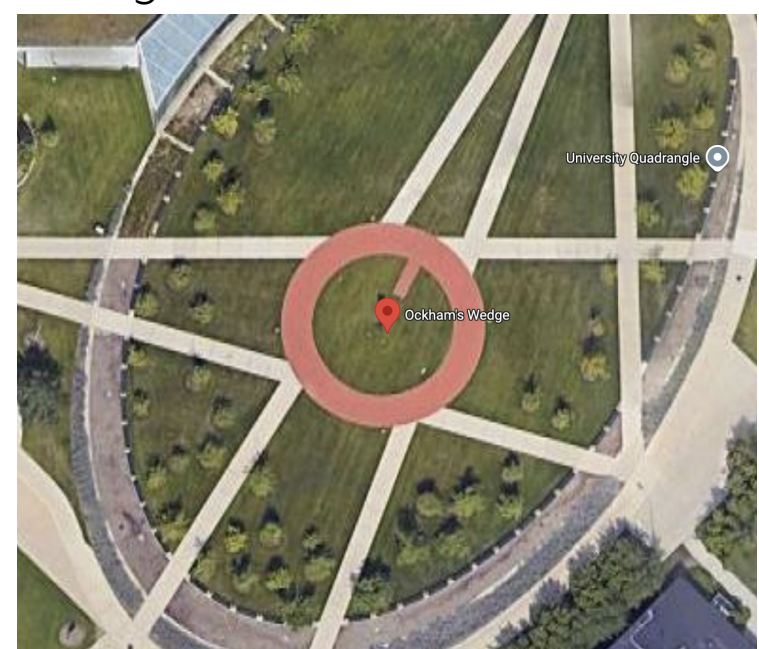
Data Collection Equipment: The LTU ACTor autonomous driving platform, integrated with the Robot Operating System (ROS), was employed for data collection. A rosbag file, a standard ROS format for recording sensor data streams, captured images from the vehicle's forward-facing camera and the corresponding steering angles [4]. This setup provided synchronized visual data and control signals essential for model training.

Data Collection Environment: Data was recorded along a fixed, predefined path: the red brick path surrounding the Ockham's Wedge at LTU. To introduce variability and ensure model robustness:

- Driving sessions included both clockwise and counterclockwise paths.
- Variations such as smooth, steady driving versus zigzag maneuvers were performed.
- Driving along the inner and outer edges of the path simulated diverse spatial alignments.

Data Extraction and Classification: A custom script was developed to extract data from the rosbag files:

- Images were saved at 200ms intervals, and file names included timestamps and steering angle labels.
- Each image was cropped to exclude irrelevant features (e.g., sky and surrounding buildings)
- The final dataset comprised of 2957 images (split into 70% Train: 2069 + 20% Val: 592 + 10% Test: 296)



RESEARCH GOALS

1. Use the (n+m) Evolutionary Strategy (ES) with the 1/5 success rule to automate hyper parameter tuning.
2. Minimize the size of the CNN model (parameters and memory footprint) while maintaining satisfactory performance for steering angle prediction. [2]
3. Validate the applicability of ES-optimized models in real-world autonomous systems.

METHODS

Model Training and Establishing a Baseline Model

Models are trained using the Keras API for Python on top of PyTorch backend to leverage GPU compute capacity. The models were trained on our dataset using the Mean Squared Error (MSE) loss function, which quantifies the average squared difference between predicted and actual steering angles. Lower MSE value and lower MAE value both indicate better performance. This serves as a benchmark for evaluating the improvements introduced by the evolution strategy (ES) with 1/5 success rule optimized models.

We began with a single-layer CNN as a simple baseline model. However, it quickly became evident that such a simplistic architecture lacked the capacity to produce results suitable for real-world applications. To address this, we adopted PilotNet—a pre-designed CNN developed by NVIDIA [1]. PilotNet, an early milestone in autonomous steering research, demonstrated the potential of GPU-accelerated deep learning for this domain.

Additionally, early stopping was added to stop the training instances after 4 epochs with no improvement in mean squared error metric. This prevents hyper parameters with no impact towards model improvement from consuming time and processing power.

Dataset and Hyper Parameter Management

The dataset was randomly divided into 70% training, 20% validation, and 10% testing splits to ensure sufficient data for model evaluation. Initially, hyper parameters included individual layer units, batch sizes, learning rates, activation functions, and optimizer selection. The layer units ranged from 20% of the baseline to 300% as baseline to allow for a wide range of optimization in each layer.

However, after initial experiments, it was observed that reducing the optimization scope to focus solely on layer units significantly improved performance and reduced computation time. Consequently, batch size (32), learning rate (0.001), activation function (ReLU), and optimizer (Adam) were fixed to match the baseline model settings.

Optimization Framework

The CNN hyper parameters—such as the number of filters in convolutional layers and the number of neurons in fully connected layers—were encoded as genes in an evolutionary strategy (ES) framework [6]. Specifically, we employed the (N+M)-ES approach, which iteratively optimizes hyper parameters by combining the N best parent models and M offspring models generated through mutations with 1/5 rule.

The process proceeds as follows:

1. **Initialization:** Generate and train N parent models with random hyper parameters within predefined ranges.
2. **Child Generation:** Create M offspring by mutating parent hyper parameters with:
$$\hat{h} = h + \text{gauss}(\sigma * (\max(h) - \min(h)))$$
where h represents a hyper parameter and σ is the step size
3. **Training and Evaluation:** Train all child models and evaluate their validation loss and accuracy.
4. **Selection:** Retain the top N models (from parents and children) based on validation loss and accuracy to form the next generation's parents.

This process iterates until the mutation step size (σ) is dynamically optimized using the 1/5 success rule:

$$\sigma_{t+1} = \begin{cases} \alpha \cdot \sigma_t & \text{if success rate} > 1/5, \\ \beta \cdot \sigma_t & \text{if success rate} < 1/5, \end{cases}$$

where α increases the step size, β decreases it based on success rate—the fraction of offspring outperforming their parents. This ensures efficient exploration of the hyper parameter space while avoiding local minima.

Computational Resources

Training was performed on Lawrence Technological University's NVIDIA A100 GPU server, enabling efficient parallel training of N parents and M offsprings model pools.

Using a larger pool of parents and offsprings allows for the algorithm to explore the solution space nearby the current parameters. The larger N and M are, the faster the evolutionary search can find a viable solution. This enables an increase in N and M values as model size decreases. The 80GB VRAM of this GPU allows using higher N and M values to balance the tradeoff between exploration and exploitation. We used N-M values in the range of 1-1 to 10-80 depending on the model size and VRAM available.

EXPERIMENTAL RESULTS

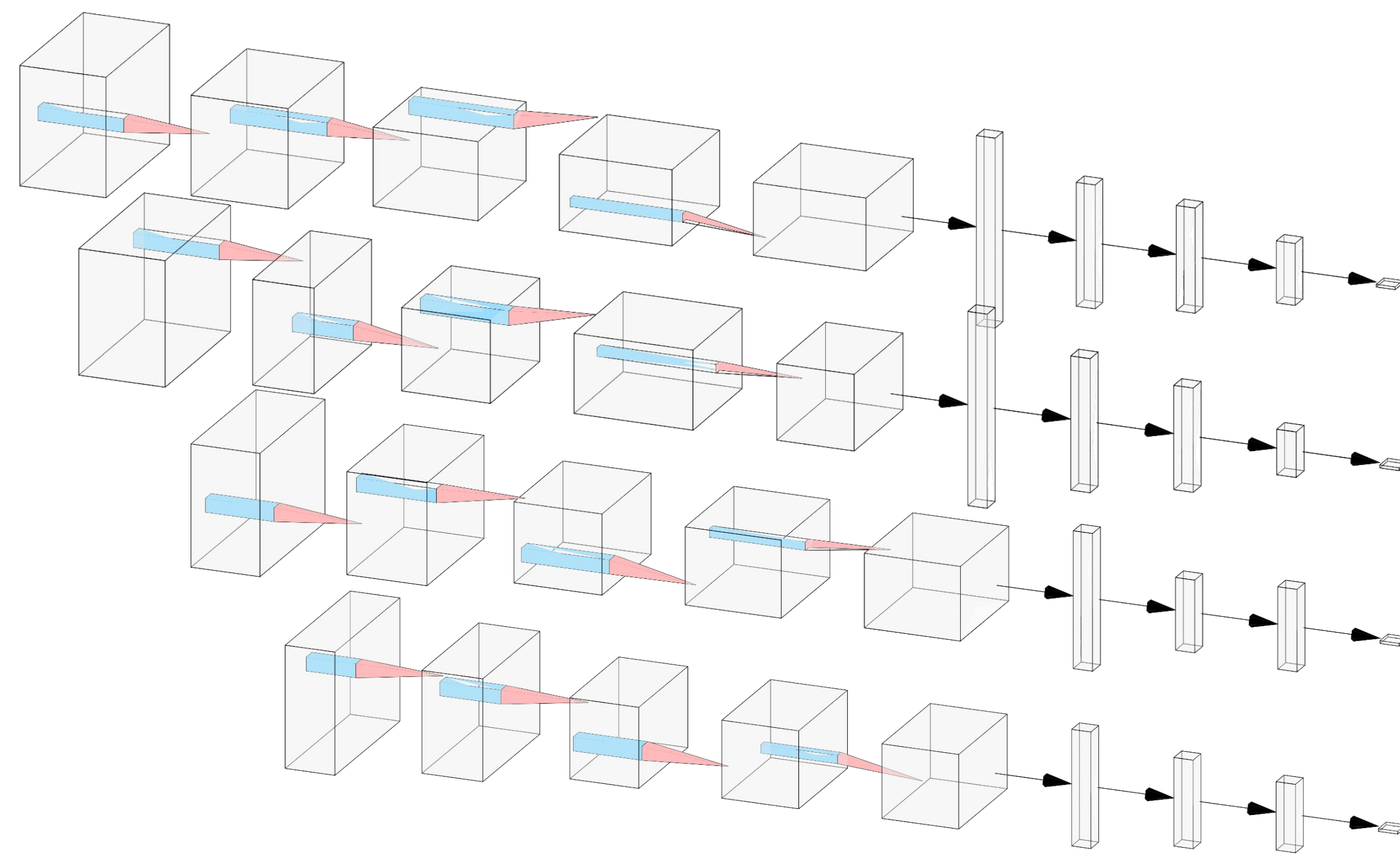


Figure 1: Model Architectures (Corresponds to the table on the right)

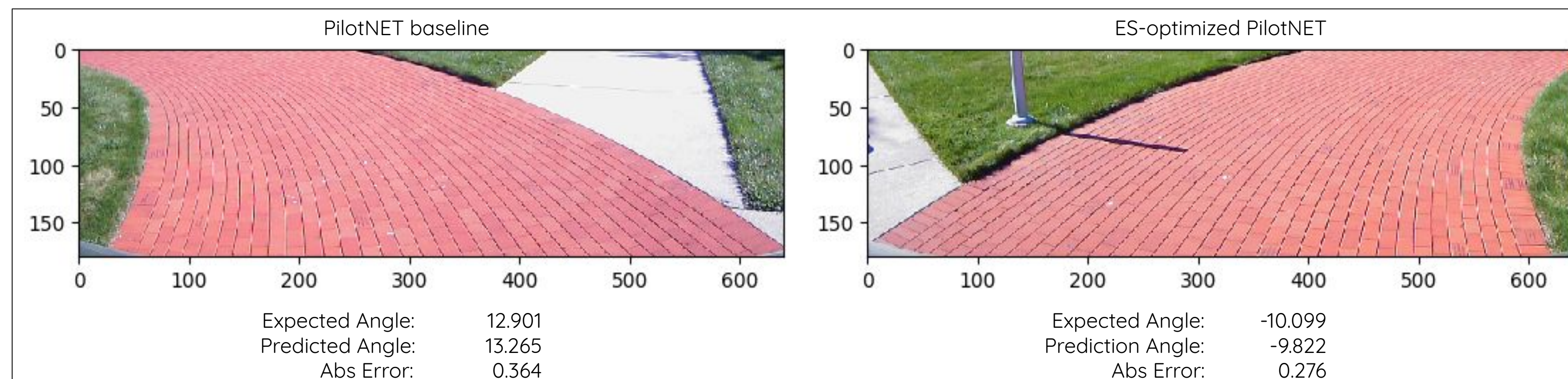


Figure 2: Prediction Results from the baseline model (left) and ES-optimized model (right)

Model	Model architecture with ES based hyper parameters	Loss (Mean Squared Error - degrees*2)	Error (Mean Absolute Error - degrees)	Model size (Million parameters)	Model size (MB)
PilotNET with baseline Convolutional and Dense architecture	CNN: 24 > 36 > 48 > 64 > 64 > DNN: 1164 > 100 > 50 > 10 > 1	1.01	0.63	245.40	936.44
Convolutional layers and Dense layers with ES	CNN: 6 > 9 > 43 > 28 > 61 > DNN: 2328 > 146 > 100 > 20 > 1	0.49	0.41	50.97	194.45
Half-size Convolutional layers and ES based Dense layers	CNN: 12 > 18 > 24 > 32 > 32 > DNN: 149 > 14 > 20 > 1	1.33	0.78	15.75	60.16
Quarter-size Convolutional layers and ES based Dense layers	CNN: 6 > 9 > 12 > 16 > 16 > DNN: 8 > 3 > 1	1.88	0.88	3.66	13.96

Table 1: Evolutionary Strategy based Hyper Parameter Optimization Comparison

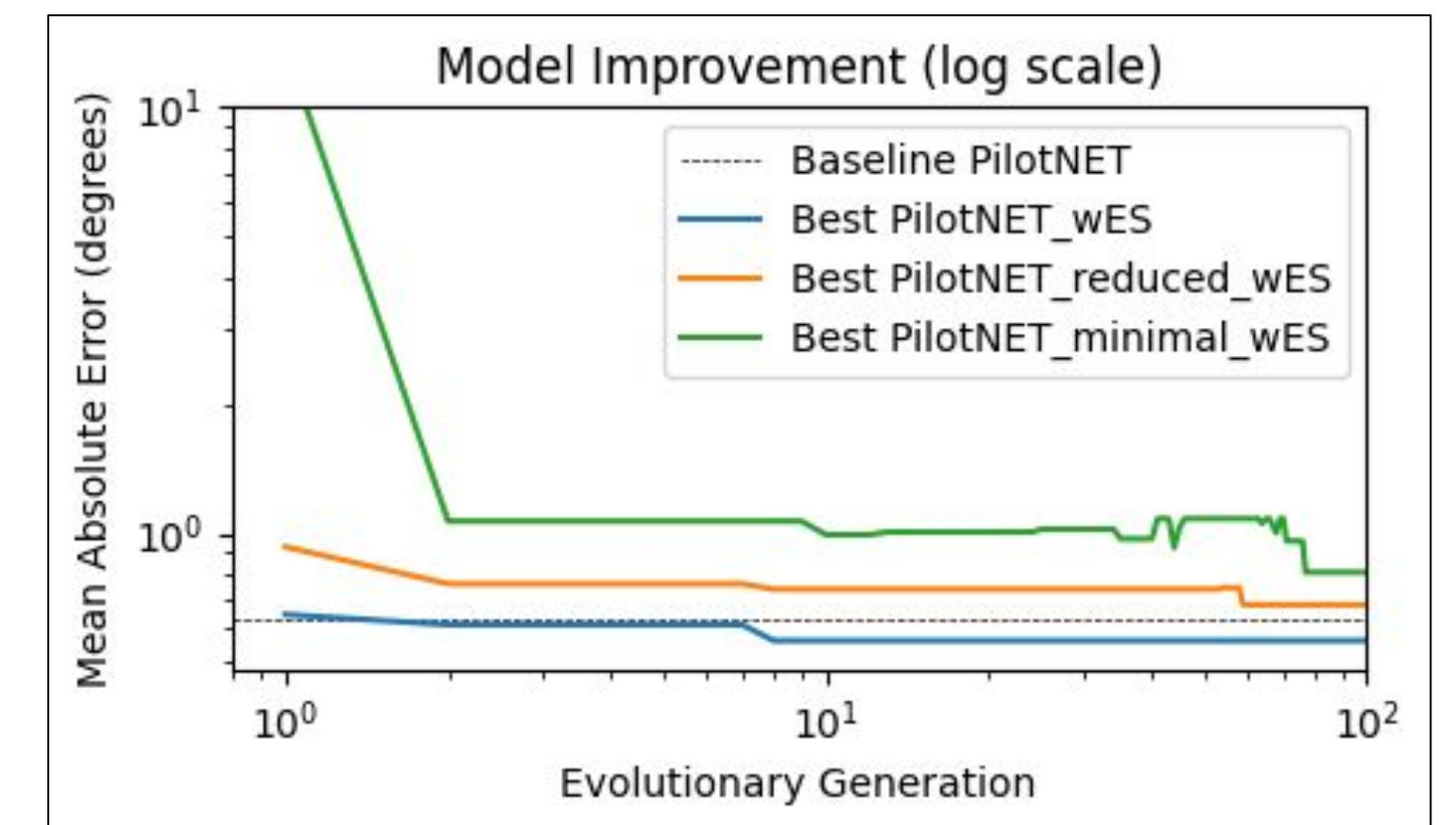


Figure 3: Model improvement over evolutionary generations

DISCUSSION

The results demonstrate the effectiveness of using evolutionary strategies (ES) with 1/5 success rule to optimize CNN architectures for steering angle prediction in autonomous driving systems and hence meets our goals. Key findings include:

Baseline Performance: The PilotNET baseline achieved reasonable accuracy but required significant computational resources due to its large model size (245.40 million parameters, 936.44 MB). This highlights the need for architecture optimization to improve efficiency without sacrificing performance.

Impact of ES Optimization: The ES-optimized PilotNET model achieved the lowest error (MSE: 0.49, MAE: 0.41 degrees) while significantly reducing the model size (50.97 million parameters, 194.45 MB). Models with reduced convolutional layers showed trade-offs: while they consumed fewer resources, their prediction accuracy relatively suffered, with the quarter-size model exhibiting the highest error (MSE: 1.88, MAE: 0.88 degrees). This study shows that using ES to optimize models for better performance or size reduction while does yield practical results that can improve application efficiency.

Real-time Performance: Visual results confirm that the optimized models deliver accurate steering predictions even in complex driving scenarios, with low absolute errors between expected and predicted steering angles. In real-world applications a small difference in error (<1.0 degree) with a major difference in model size (5-25% of baseline size) is a valid tradeoff.

In the future, we plan to:

- Expand the dataset by recording driving data on various roads across the LTU campus, including different surfaces, path geometries, and lighting conditions.
- Add Recurrent Neural Network layers to train models based on the past steering inputs [5]
- Deploy the optimized CNN models on the LTU ACTor autonomous driving platform.
- Evaluate optimized models on low-power compute platforms, such as NVIDIA Jetson or Raspberry Pi, to assess scalability for cost-effective systems.

Find future updates at <https://github.com/Aeolus96/LTU-Self-Drive-Sim>

REFERENCES

1. Bojarski, M., Del Testa, D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., ... Zieba, K. (2016). End to End Learning for Self-Driving Cars. Retrieved from <https://arxiv.org/abs/1604.07316>
2. Salimans, T., Ho, J., Chen, X., Sidor, S., & Sutskever, I. (2017). Evolution Strategies as a Scalable Alternative to Reinforcement Learning. Retrieved from <https://arxiv.org/abs/1703.03864>
3. J. Chung, I. Bae, J. Moon, T. Kim, J. Kim and S. Kim, "End-to-End Steering Controller with CNN-based Closed-loop Feedback for Autonomous Vehicles," 2018 IEEE Intelligent Vehicles Symposium (IV), Changshu, China, 2018, pp. 617-622, doi: 10.1109/IVS.2018.8500440.
4. T. Qiu and Z. Huang, "Learning A Steering Decision Policy for End-to-End Control of Autonomous Vehicle," 2019 5th International Conference on Control, Automation and Robotics (ICCAR), Beijing, China, 2019, pp. 347-351, doi: 10.1109/ICCAR.2019.8813428.
5. Valiente, R., Zaman, M., Ozer, S., & Fallah, Y. P. (2019). Controlling Steering Angle for Cooperative Self-driving Vehicles utilizing CNN and LSTM-based Deep Networks. Retrieved from <https://arxiv.org/abs/1904.04375>
6. C.-J. Chung and R. Reynolds, "Knowledge-based self-adaptation in evolutionary search," International Journal of Pattern Recognition and Artificial Intelligence, vol. 14, no. 01, pp. 19-33, 2000.